

Decision Tree & Random Forest Classifier

Study-Note Style Machine Learning Notebook

Python	scikit-learn	Seaborn	Status
3.x	Classification	Iris + Tips	Complete

Note: This PDF follows the same study-note approach as the uploaded Logistic Regression reference: each major step explains what the code is doing and why it is needed, followed by the working code and representative output.

Datasets: Iris and Tips (Seaborn sample datasets)

Goal: Build, evaluate, and compare a Decision Tree Classifier and a Random Forest Classifier on both datasets.

Author: Sambit7848

What this notebook covers

Section	Topic
1	Machine Learning classification & core terms
2	Decision Tree — concept and working
3	Random Forest — concept and working
4	Part A: Iris Dataset
5	Part B: Tips Dataset
6	Final comparison & key takeaways

Section 1: Core Notes Before Modeling

Note: What is Classification?

Note: Classification is a supervised machine learning task in which the model learns from labeled examples and predicts a category. In this notebook, Iris has 3 classes (setosa, versicolor, virginica) and Tips has 2 classes (Lunch/Dinner after encoding).

Note: What is a Decision Tree?

Note: A Decision Tree is a flowchart-like model that repeatedly splits the data using feature-based rules. Each internal node asks a question, each branch represents an outcome, and a leaf gives the final prediction. The model is easy to visualize and explain.

Note: What is a Random Forest?

Note: Random Forest is an ensemble method that builds many decision trees and combines their predictions. Each tree is trained using a random sample of the training data and a random subset of features at split time. Combining trees usually makes the model more stable and less sensitive to one particular tree.

Decision Tree vs Random Forest

Aspect	Decision Tree	Random Forest
Structure	One tree	Many trees
Main idea	Sequential feature splits	Ensemble of independent trees
Overfitting	Can overfit easily	Usually reduces overfitting
Interpretability	High	Lower than one tree
Stability	Sensitive to data changes	Generally more robust
Training cost	Lower	Higher

Note: Entropy

Note: Entropy measures the impurity or uncertainty in a node. A pure node contains mostly one class and has low entropy. The notebook uses `criterion='entropy'`, so the tree chooses splits that reduce impurity.

Entropy formula:

$$\text{Entropy}(S) = - \sum p_i \log(p_i)$$

Note: Information Gain

Note: Information Gain measures how much a split reduces entropy. A good split creates child nodes that are more pure than the parent node. With `criterion='entropy'`, the Decision Tree and Random Forest use this idea when choosing splits.

Note: Bootstrap + Feature Randomness in Random Forest

Note: Random Forest gets its strength from randomness. Different trees see different training samples, and each split considers a randomized subset of features. Their combined vote is usually more reliable than a single tree.

Section 2: Model Evaluation Notes

Note: Accuracy Score

Note: Accuracy is the proportion of correct predictions out of all predictions. It is useful when class distributions are reasonably balanced, but it should not be the only metric in a serious classification problem.

$$\text{Accuracy} = \text{Correct Predictions} / \text{Total Predictions}$$

Note: Confusion Matrix

Note: A confusion matrix compares actual classes with predicted classes. For binary classification it contains True Positive, True Negative, False Positive, and False Negative counts. For multiclass classification, it shows the number of examples assigned from each actual class to each predicted class.

Note: Classification Report

Note: The classification report summarizes precision, recall, F1-score, and support for each class. Precision asks how many predicted positives were actually correct; recall asks how many actual positives were found; F1-score balances precision and recall.

Note: Train-Test Split

Note: The notebook uses `test_size=0.2` and `random_state=42`. Therefore, 80% of rows are used for training and 20% for testing. `random_state=42` makes the split reproducible.

```
x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.2, random_state=42
)
```

Part A: Iris Dataset

Target variable: species (3 classes)

Note: The Iris dataset contains 150 rows and 5 columns: four numeric measurements and one target column, species. The task is to predict which of the three Iris species a flower belongs to.

Q1. Import libraries and load the Iris dataset

Note: We import NumPy, Matplotlib, Pandas, Seaborn and warnings. `seaborn.load_dataset('iris')` loads the sample Iris dataset used by the original notebook.

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns
import warnings

warnings.filterwarnings('ignore')

dataset = sns.load_dataset('iris')

dataset.head()
```

sepal_length	sepal_width	petal_length	petal_width	species
5.1	3.5	1.4	0.2	setosa
4.9	3.0	1.4	0.2	setosa
4.7	3.2	1.3	0.2	setosa
4.6	3.1	1.5	0.2	setosa
5.0	3.6	1.4	0.2	setosa

Q2. Check shape, data types, and missing values

Note: Before modeling, profile the dataset. `shape` gives rows and columns, `dtypes` shows the data types, and `isnull().sum()` checks whether missing values are present.

```
print(dataset.shape)

print()

print(dataset.dtypes)

print()

print(dataset.isnull().sum())
```

```
(150, 5)

sepal_length    float64
sepal_width     float64
petal_length    float64
petal_width     float64
species         object

All columns: 0 missing values
```

Q3. Define X and y

Note: X contains the independent variables used to make predictions. y contains the dependent variable, species, which the model must learn to predict.

```
X = dataset.drop('species', axis=1)

y = dataset['species']
```

Q4. Split into training and testing sets

Note: The model learns from `X_train` and `y_train`, while `X_test` and `y_test` are held back for an unbiased evaluation.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

Part A — Decision Tree Classifier on Iris

Q5. Build and train the Decision Tree

Note: DecisionTreeClassifier creates a tree-based classifier. criterion='entropy' tells the model to use entropy/information gain when selecting splits. random_state=42 makes the result reproducible.

```
from sklearn.tree import DecisionTreeClassifier

dt = DecisionTreeClassifier(
    criterion='entropy',
    random_state=42
)

dt.fit(X_train, y_train)
```

Q6. Predict the species for the test set

Note: predict() sends unseen test rows through the trained tree and returns the predicted class for each row.

```
y_pred = dt.predict(X_test)

y_pred
```

```
['versicolor' 'setosa' 'virginica' 'versicolor' 'versicolor' ...]
```

Q7. Generate the Classification Report

Note: The report shows precision, recall, F1-score, and support for each Iris class.

```
from sklearn.metrics import classification_report

print(classification_report(y_test, y_pred))
```

Class	Precision	Recall	F1-score	Support
setosa	1.00	1.00	1.00	10
versicolor	1.00	1.00	1.00	9
virginica	1.00	1.00	1.00	11
accuracy			1.00	
macro	avg	1.00	1.00	1.00
weighted	avg	1.00	1.00	1.00

Q8. Generate the Confusion Matrix

Note: Rows represent the actual class and columns represent the predicted class. A perfect diagonal means every test example was classified correctly.

```
from sklearn.metrics import confusion_matrix

cm = confusion_matrix(y_test, y_pred)

print(cm)
```

Actual \ Predicted	setosa	versicolor	virginica
setosa	10	0	0
versicolor	0	9	0
virginica	0	0	11

Q9. Calculate Accuracy

Note: For this 80/20 Iris split, the Decision Tree correctly classifies all 30 test samples.

```
from sklearn.metrics import accuracy_score  
  
accuracy = accuracy_score(y_test, y_pred)  
  
print("Accuracy:", accuracy)
```

```
Accuracy: 1.0
```

Part A — Random Forest Classifier on Iris

Q10. Build and train the Random Forest

Note: RandomForestClassifier creates an ensemble of decision trees. `n_estimators=10` means the notebook builds 10 trees. `criterion='entropy'` keeps the same impurity criterion used by the Decision Tree.

```
from sklearn.ensemble import RandomForestClassifier

rfc = RandomForestClassifier(
    n_estimators=10,
    criterion='entropy',
    random_state=42
)

rfc.fit(X_train, y_train)
```

Q11. Predict the species

Note: The trained forest combines the predictions of its trees to decide the final class.

```
y_pred = rfc.predict(X_test)

y_pred

['versicolor' 'setosa' 'virginica' 'versicolor' 'versicolor' ...]
```

Q12. Classification Report

```
from sklearn.metrics import classification_report

print(classification_report(y_test, y_pred))
```

Class	Precision	Recall	F1-score	Support
setosa	1.00	1.00	1.00	10
versicolor	1.00	1.00	1.00	9
virginica	1.00	1.00	1.00	11
accuracy			1.00	
macro	avg	1.00	1.00	1.00
weighted	avg	1.00	1.00	1.00

Q13. Confusion Matrix

```
from sklearn.metrics import confusion_matrix

cm = confusion_matrix(y_test, y_pred)

print(cm)
```

Actual \ Predicted	setosa	versicolor	virginica
setosa	10	0	0
versicolor	0	9	0
virginica	0	0	11

Q14. Accuracy Score

```
from sklearn.metrics import accuracy_score

accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)
```

```
Accuracy: 1.0
```

Q15. Compare Decision Tree and Random Forest on Iris

Note: Both models achieve the same accuracy on this particular test split, so the difference is 0.0.

```
dt_accuracy = dt.score(X_test, y_test)
rf_accuracy = rfc.score(X_test, y_test)

print("Decision Tree Accuracy :", dt_accuracy)
print("Random Forest Accuracy :", rf_accuracy)
print("Difference :", rf_accuracy - dt_accuracy)

if rf_accuracy > dt_accuracy:
    print("Random Forest performed better.")
elif rf_accuracy < dt_accuracy:
    print("Decision Tree performed better.")
else:
    print("Both models performed equally.")
```

```
Decision Tree Accuracy : 1.0
Random Forest Accuracy : 1.0
Difference : 0.0
Both models performed equally.
```

Part B: Tips Dataset

Target variable: time (2 classes — Lunch/Dinner)

Note: The Tips dataset contains 244 rows and 7 columns. The target is time. Before modeling, the categorical columns are converted into numeric values using LabelEncoder.

Q16. Load the Tips dataset

Note: The same Seaborn loading approach is used here. The first five rows are displayed to understand the structure before preprocessing.

```
import seaborn as sns

dataset = sns.load_dataset('tips')

dataset.head()
```

total_bill	tip	sex	smoker	day	time	size
16.99	1.01	Female	No	Sun	Dinner	2
10.34	1.66	Male	No	Sun	Dinner	3
21.01	3.5	Male	No	Sun	Dinner	3
23.68	3.31	Male	No	Sun	Dinner	2
24.59	3.61	Female	No	Sun	Dinner	4

Q17. Check shape, data types, and missing values

Note: The dataset has 244 rows and 7 columns. The original categorical columns are stored as object values and there are no missing values.

```
print(dataset.shape)

print()

print(dataset.dtypes)

print()

print(dataset.isnull().sum())
```

```
(244, 7)

total_bill    float64
tip           float64
sex           object
smoker        object
day           object
time          object
size          int64

All columns: 0 missing values
```

Q18. Encode categorical columns

Note: Most scikit-learn classifiers expect numeric inputs. LabelEncoder converts each categorical column into integer labels. For this dataset: sex → Female=0/Male=1, smoker → No=0/Yes=1, day → Fri=0/Sat=1/Sun=2/Thur=3, and time → Dinner=0/Lunch=1.

```

from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()

dataset['sex'] = le.fit_transform(dataset['sex'])
dataset['smoker'] = le.fit_transform(dataset['smoker'])
dataset['day'] = le.fit_transform(dataset['day'])
dataset['time'] = le.fit_transform(dataset['time'])

dataset.head()

```

total_bill	tip	sex	smoker	day	time	size
16.99	1.01	0.0	0.0	2.0	0.0	2.0
10.34	1.66	1.0	0.0	2.0	0.0	3.0
21.01	3.5	1.0	0.0	2.0	0.0	3.0
23.68	3.31	1.0	0.0	2.0	0.0	2.0
24.59	3.61	0.0	0.0	2.0	0.0	4.0

Q19. Define X and y

Note: X contains all columns except time. y contains time, which is the target to predict.

```

X = dataset.drop('time', axis=1)

y = dataset['time']

```

Q20. Split the data

Note: The same 80/20 split and random seed are used so the evaluation is reproducible.

```

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

```

Part B — Decision Tree Classifier on Tips

Q21. Build and train the Decision Tree

Note: The same Decision Tree configuration is reused: entropy criterion and random_state=42.

```
from sklearn.tree import DecisionTreeClassifier

dt = DecisionTreeClassifier(
    criterion='entropy',
    random_state=42
)

dt.fit(X_train, y_train)
```

Q22. Predict time for the test set

Note: The trained tree predicts the encoded target class for each unseen test row.

```
y_pred = dt.predict(X_test)

y_pred
```

First predictions: [0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 1 0 0 0 0 ...]

Q23. Classification Report

```
from sklearn.metrics import classification_report

print(classification_report(y_test, y_pred))
```

Class	Precision	Recall	F1-score	Support
accuracy			0.94	
macro	avg	0.94	0.92	0.93
weighted	avg	0.94	0.94	0.94

Q24. Confusion Matrix

Note: For the binary target, the matrix shows how many Lunch/Dinner records were correctly or incorrectly classified.

```
from sklearn.metrics import confusion_matrix

cm = confusion_matrix(y_test, y_pred)

print(cm)
```

Actual \ Predicted	Dinner (0)	Lunch (1)
Dinner (0)	33	1
Lunch (1)	2	13

Q25. Accuracy Score

Note: The Decision Tree reaches about 93.88% accuracy on the test split.

```
from sklearn.metrics import accuracy_score

accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)
```

Accuracy: 0.9387755102040817

Part B — Random Forest Classifier on Tips

Q26. Build and train the Random Forest

Note: The forest contains 10 decision trees. Each tree contributes to the final prediction, making the ensemble more stable than relying on one tree.

```
from sklearn.ensemble import RandomForestClassifier

rfc = RandomForestClassifier(
    n_estimators=10,
    criterion='entropy',
    random_state=42
)

rfc.fit(X_train, y_train)
```

Q27. Predict time

Note: The forest predicts the encoded target for the test rows.

```
y_pred = rfc.predict(X_test)

y_pred
```

First predictions: [0 0 0 0 0 1 0 0 0 0 0 0 0 0 1 0 0 0 0 ...]

Q28. Classification Report

```
from sklearn.metrics import classification_report

print(classification_report(y_test, y_pred))
```

Class	Precision	Recall	F1-score	Support
accuracy			0.96	
macro	avg	0.94	0.97	0.95
weighted	avg	0.96	0.96	0.96

Q29. Confusion Matrix

```
from sklearn.metrics import confusion_matrix

cm = confusion_matrix(y_test, y_pred)

print(cm)
```

Actual \ Predicted	Dinner (0)	Lunch (1)
Dinner (0)	32	2
Lunch (1)	0	15

Q30. Accuracy Score

Note: The Random Forest reaches about 95.92% accuracy on the same test split.

```
from sklearn.metrics import accuracy_score

accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)
```

Accuracy: 0.9591836734693877

Q31. Compare Decision Tree and Random Forest on Tips

Note: Random Forest performs better on this split by approximately 0.0204, or 2.04 percentage points.

```
dt_accuracy = dt.score(X_test, y_test)
rf_accuracy = rfc.score(X_test, y_test)

print("Decision Tree Accuracy :", dt_accuracy)
print("Random Forest Accuracy :", rf_accuracy)
print("Difference :", rf_accuracy - dt_accuracy)

if rf_accuracy > dt_accuracy:
    print("Random Forest performed better.")
elif dt_accuracy > rf_accuracy:
    print("Decision Tree performed better.")
else:
    print("Both models performed equally.")
```

```
Decision Tree Accuracy : 0.9387755102040817
Random Forest Accuracy : 0.9591836734693877
Difference : 0.020408163265306034
Random Forest performed better.
```

Section 3: Final Comparison

Q32. Summarize all four model results

Note: The final table compares the two algorithms across both datasets using the same 80/20 train-test strategy and `random_state=42`. The highest accuracy is achieved by both Iris models at 1.00 on this test split.

Model	Accuracy
Decision Tree - Iris	1.0000 (100.00%)
Random Forest - Iris	1.0000 (100.00%)
Decision Tree - Tips	0.9388 (93.88%)
Random Forest - Tips	0.9592 (95.92%)

Overall result

Note: The Iris Decision Tree and Iris Random Forest both achieved 100% accuracy on the selected 30-row test split. On the Tips dataset, Random Forest outperformed the Decision Tree: 95.92% versus 93.88%.

Key Takeaways

1. Decision Tree is a simple, interpretable model that makes predictions through a sequence of feature-based splits.
2. Random Forest combines multiple decision trees and usually gives better generalization and stability.
3. `criterion='entropy'` uses entropy-based impurity reduction when selecting splits.
4. `n_estimators=10` means the Random Forest contains 10 decision trees in this notebook.
5. `train_test_split` with `test_size=0.2` keeps 20% of the data for testing.
6. `random_state=42` makes the train-test split and model randomness reproducible.
7. Categorical data must be encoded before it is passed to the models; the Tips section uses `LabelEncoder`.
8. Accuracy is useful, but `classification_report` and `confusion_matrix` provide more detailed information about model errors.
9. A model that performs best on one dataset is not automatically best for every dataset.

Quick Interview Notes

Question	Short Answer
What is a Decision Tree?	A tree-based classifier that splits data using feature-based rules.
What is Random Forest?	An ensemble of many decision trees whose predictions are combined.
Why use Random Forest?	It generally improves stability and reduces the risk of relying on one overfit tree.
What is entropy?	A measure of impurity/uncertainty used to choose useful splits.
What is <code>n_estimators</code> ?	The number of trees in the Random Forest.
Why encode categorical data?	Most scikit-learn models require numerical feature values.
Why use a test set?	To evaluate the trained model on unseen data.

Appendix: Notebook Flow

The uploaded notebook follows this overall workflow:

1. Load dataset
2. Inspect shape, data types, and missing values
3. Define X (features) and y (target)
4. Split into training and testing data
5. Train Decision Tree
6. Predict
7. Classification Report
8. Confusion Matrix
9. Accuracy Score
10. Train Random Forest
11. Predict
12. Classification Report
13. Confusion Matrix
14. Accuracy Score
15. Compare Decision Tree vs Random Forest
16. Repeat for the second dataset
17. Summarize all four model accuracies

Reproducibility settings used in the notebook

Setting	Value
Train/Test split	80% train / 20% test
random_state	42
Decision Tree criterion	entropy
Random Forest trees	10
Iris target	species
Tips target	time
Tips categorical encoding	LabelEncoder

Note: This PDF is designed as a GitHub-friendly study-note artifact rather than a project report. It keeps the original notebook's two-dataset structure and adds explanatory notes around the code, similar to the uploaded Logistic Regression reference.

Notebook prepared by **Sambit7848** as part of a hands-on Machine Learning portfolio.